In [39]:
```
# ======================================================================
# ENVIRONMENT SETUP
# Purpose: Import all libraries and configure notebook settings.
# Expected: All imports succeed and versions print cleanly.
# ======================================================================

import sys
import warnings
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

sys.path.insert(0, str(Path.cwd()))
from code_library.data_preparation import load_cmapss, load_naval

warnings.filterwarnings("ignore")
sns.set_theme(style="whitegrid", palette="muted")
plt.rcParams["figure.dpi"] = 120
RANDOM_SEED = 42

CMAPSS_DIR = "Data/CMaps"
NAVAL_DIR  = "Data/UCI CBM Dataset"

print("Setup complete")
print(f"numpy  : {np.__version__}")
print(f"pandas : {pd.__version__}")
```

```
Setup complete
numpy  : 1.26.4
pandas : 2.2.2
```

In [40]:
```
# ======================================================================
# DATA LOADING — CMAPSS & NAVAL
# Purpose: Load raw data from both domains into DataFrames for EDA.
# Expected: CMAPSS FD001 train ~20,631 rows x 26 cols. Naval 11,934 ro
# ======================================================================

# CMAPSS — load FD001 for initial EDA (will loop all 4 subsets later)
cmapss_train, cmapss_test = load_cmapss(subset="FD001", raw_dir=CMAPSS

# Naval — load raw (un-normalized) for EDA
naval_x, naval_y = load_naval(raw_dir=NAVAL_DIR)

print("CMAPSS FD001")
print(f"  Train shape : {cmapss_train.shape}")
print(f"  Test shape  : {cmapss_test.shape}")
print(f"  Columns     : {list(cmapss_train.columns)}")
print()
```

```
print("Naval Propulsion Plants")
print(f"  Features shape : {naval_x.shape}")
print(f"  Targets shape  : {naval_y.shape}")
print(f"  Features       : {list(naval_x.columns)}")
print(f"  Targets        : {list(naval_y.columns)}")
```

```
CMAPSS FD001
  Train shape : (20631, 21)
  Test shape  : (13096, 21)
  Columns     : ['unit_id', 'cycle', 'op_setting_1', 'op_setting_2', 'o
p_setting_3', 's2', 's3', 's4', 's6', 's7', 's8', 's9', 's11', 's12', '
s13', 's14', 's15', 's17', 's20', 's21', 'rul']

Naval Propulsion Plants
  Features shape : (11934, 16)
  Targets shape  : (11934, 2)
  Features       : ['lever_position', 'ship_speed', 'gt_shaft_torque',
'gt_rate_of_revolutions', 'gas_generator_rate_of_revolutions', 'starboa
rd_propeller_torque', 'port_propeller_torque', 'hp_turbine_exit_tempera
ture', 'gt_compressor_inlet_air_temperature', 'gt_compressor_outlet_air
_temperature', 'hp_turbine_exit_pressure', 'gt_compressor_inlet_air_pre
ssure', 'gt_compressor_outlet_air_pressure', 'gt_exhaust_gas_pressure',
'turbine_injection_control', 'fuel_flow']
  Targets        : ['kMc', 'kMt']
```

In [41]:

```
# ======================================================================
# CMAPSS FD001 — BASIC STATISTICS
# Purpose: Understand the scale, range, and distribution of sensor rea
#          and the RUL target before plotting.
# Expected: 20 columns of stats. RUL min=0, max=125 (clipped). Sensors
#           normalized 0-1 so min~0, max~1.
# ======================================================================

print("=== CMAPSS FD001 — Training Set Statistics ===\n")
print(f"Unique engines : {cmapss_train['unit_id'].nunique()}")
print(f"Total cycles   : {len(cmapss_train)}")
print(f"RUL range      : {cmapss_train['rul'].min():.0f} — {cmapss_tra
print(f"Avg engine life: {cmapss_train.groupby('unit_id')['cycle'].max
print()
cmapss_train.describe().round(3)
```

```
=== CMAPSS FD001 — Training Set Statistics ===

Unique engines : 100
Total cycles   : 20631
RUL range      : 0 — 125
Avg engine life: 206.3 cycles
```

Out[41]:

	unit_id	cycle	op_setting_1	op_setting_2	op_setting_3	
count	20631.000	20631.000	20631.000	20631.000	20631.0	20631.0
mean	51.507	108.808	-0.000	0.000	100.0	0.4
std	29.228	68.881	0.002	0.000	0.0	0.
min	1.000	1.000	-0.009	-0.001	100.0	0.0
25%	26.000	52.000	-0.002	-0.000	100.0	0.3
50%	52.000	104.000	0.000	0.000	100.0	0.4
75%	77.000	156.000	0.002	0.000	100.0	0.5
max	100.000	362.000	0.009	0.001	100.0	1.0

8 rows × 21 columns

In [42]:

```
# ======================================================================
# NAVAL — BASIC STATISTICS
# Purpose: Understand the scale and distribution of Naval features and
#          degradation targets before plotting.
# Expected: 11,934 rows. kMc and kMt both near 1.0 (healthy state),
#           with values decreasing toward ~0.95 as degradation increas
# ======================================================================

print("=== Naval Propulsion Plants — Statistics ===\n")
print(f"Total records : {len(naval_x)}")
print(f"Features      : {naval_x.shape[1]}")
print(f"Targets       : {naval_y.shape[1]}")
print()
print("--- Target ranges ---")
print(naval_y.describe().round(6))
print()
print("--- Feature ranges ---")
naval_x.describe().round(3)
```

```
=== Naval Propulsion Plants — Statistics ===

Total records : 11934
Features      : 16
Targets       : 2

--- Target ranges ---
               kMc         kMt
count  11934.00000  11934.0000
mean       0.97500      0.9875
std        0.01472      0.0075
min        0.95000      0.9750
25%        0.96200      0.9810
50%        0.97500      0.9875
75%        0.98800      0.9940
max        1.00000      1.0000

--- Feature ranges ---
```

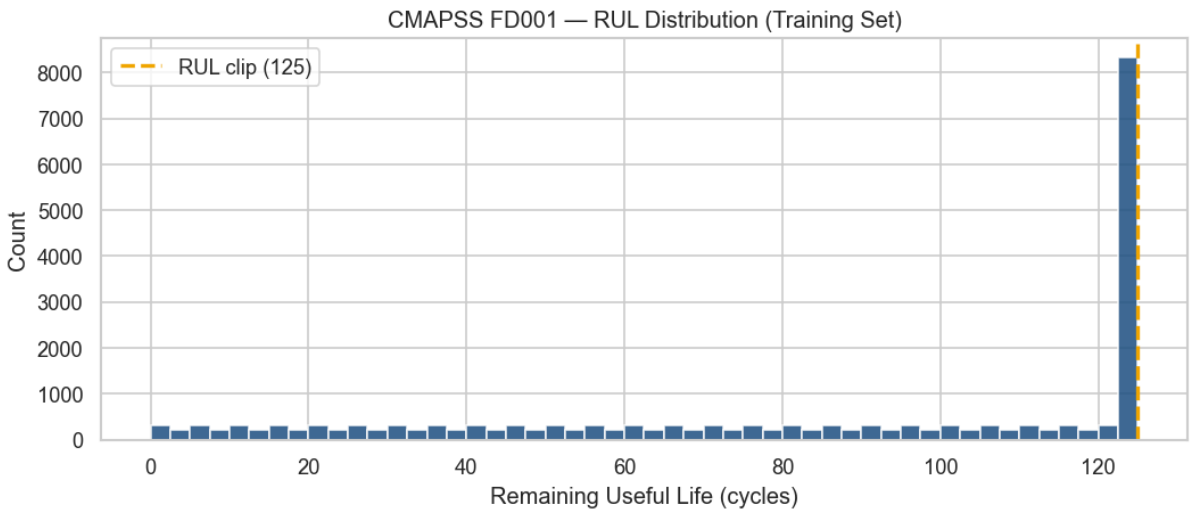
Out[42]:

	lever_position	ship_speed	gt_shaft_torque	gt_rate_of_revolutions	gas
count	11934.000	11934.000	11934.000	11934.000	
mean	5.167	15.000	27247.499	2136.289	
std	2.626	7.746	22148.613	774.084	
min	1.138	3.000	253.547	1307.675	
25%	3.144	9.000	8375.884	1386.758	
50%	5.140	15.000	21630.659	1924.326	
75%	7.148	21.000	39001.427	2678.079	
max	9.300	27.000	72784.872	3560.741	

In [43]:

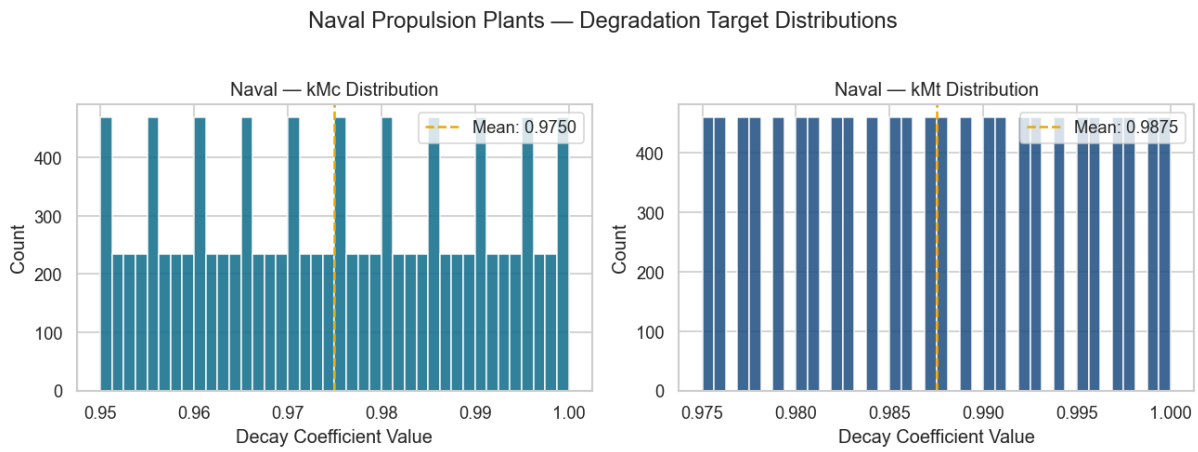
```
# ======================================================================
# EDA — CMAPSS: RUL DISTRIBUTION
# Purpose: Visualize the distribution of RUL values across all trainin
#          engines to understand the shape of the prediction target.
# Expected: Right-skewed histogram with a hard cutoff at 125 (RUL clip
#           Most engines have RUL concentrated between 0-125.
# ======================================================================

fig, ax = plt.subplots(figsize=(9, 4))
ax.hist(cmapss_train["rul"], bins=50, color="#1C4E80", edgecolor="whit
ax.axvline(x=125, color="#F0A500", linestyle="--", linewidth=2, label=
ax.set_xlabel("Remaining Useful Life (cycles)")
ax.set_ylabel("Count")
ax.set_title("CMAPSS FD001 — RUL Distribution (Training Set)")
ax.legend()
plt.tight_layout()
plt.savefig("images/cmapss_rul_distribution.png", dpi=120)
```

```
plt.show()
```

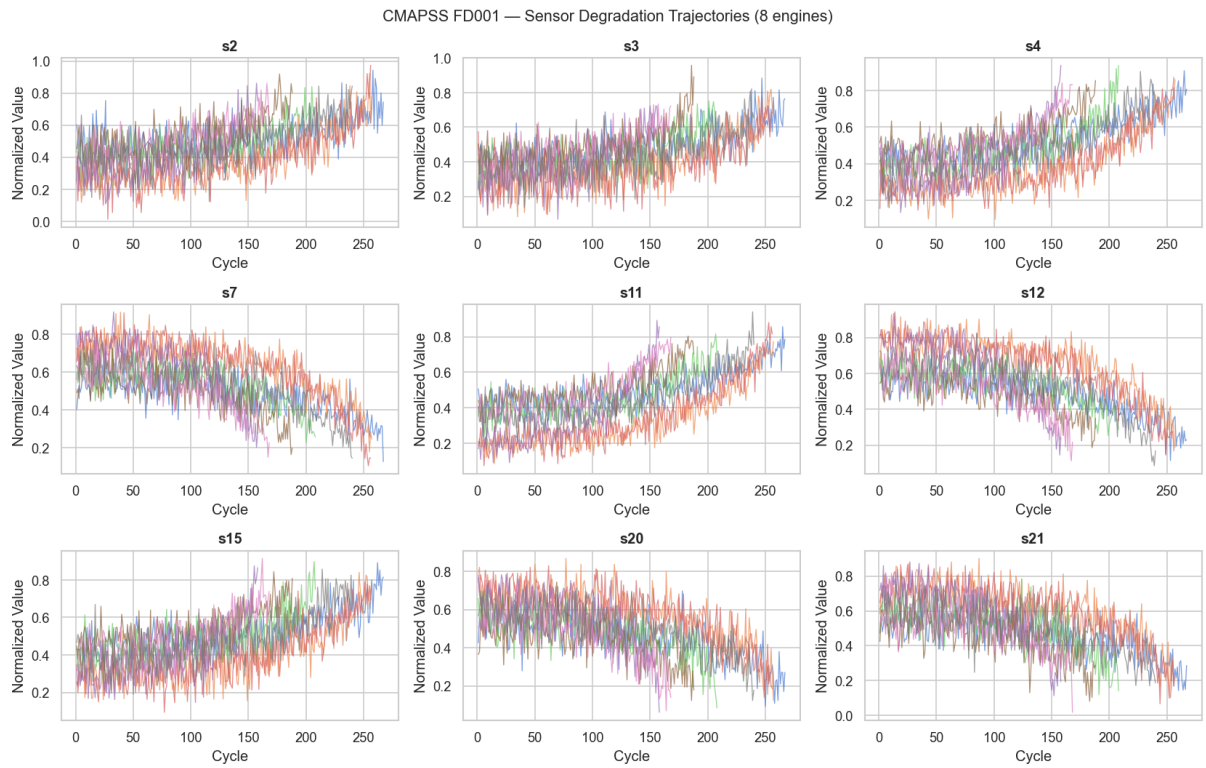


In [44]:

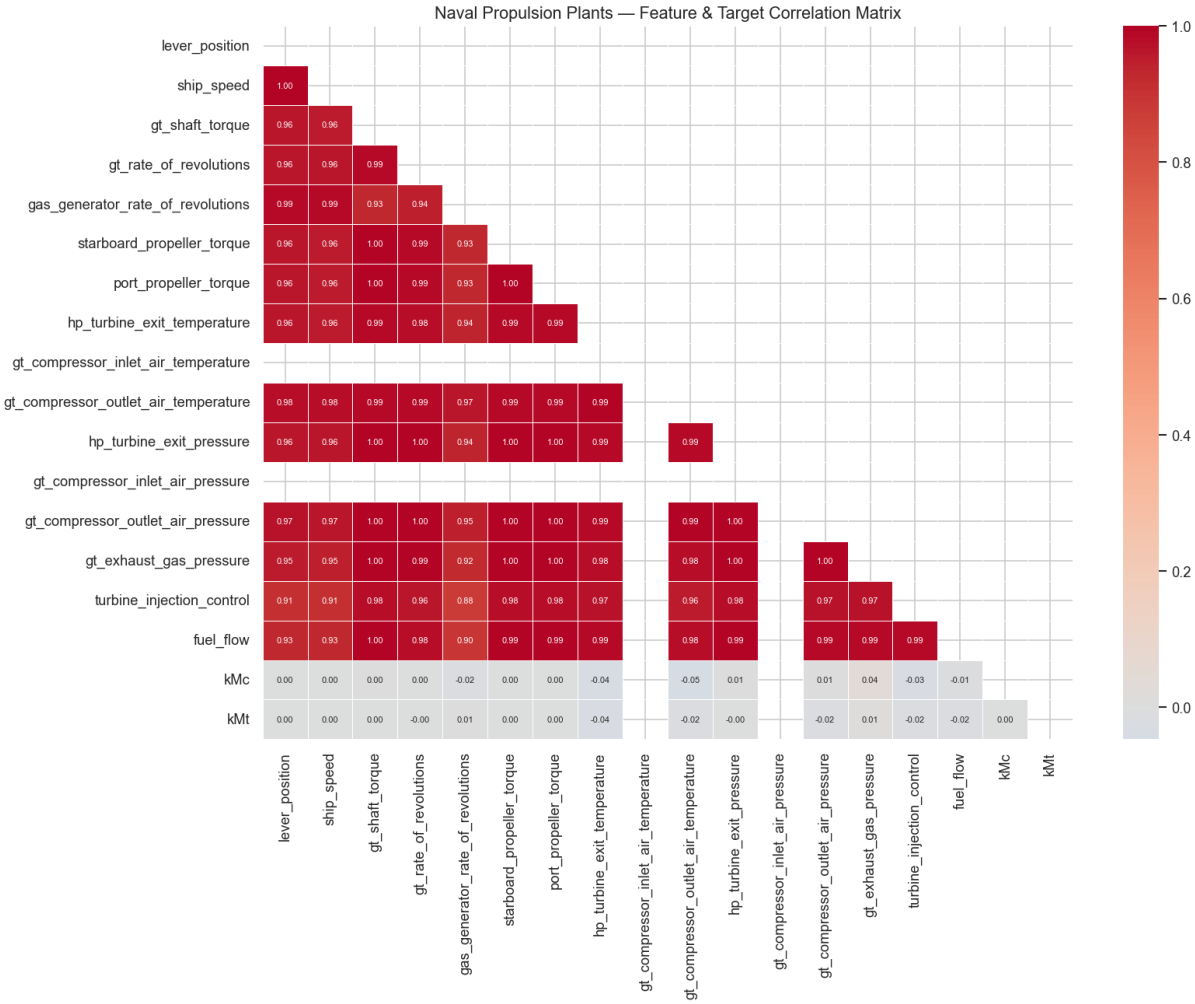
```
# ======================================================================
# EDA — NAVAL: TARGET DISTRIBUTIONS (kMc and kMt)
# Purpose: Visualize the distribution of both degradation targets side
#          side to understand their range and spread before modeling.
# Expected: Both targets concentrated near 1.0 (healthy), spreading le
#           toward degraded values. kMt has a narrower range than kMc.
# ======================================================================

fig, axes = plt.subplots(1, 2, figsize=(11, 4))
colors = ["#0E6B8A", "#1C4E80"]

for ax, col, color in zip(axes, naval_y.columns, colors):
    ax.hist(naval_y[col], bins=40, color=color, edgecolor="white", alp
    ax.set_title(f"Naval — {col} Distribution")
    ax.set_xlabel("Decay Coefficient Value")
    ax.set_ylabel("Count")
    ax.axvline(naval_y[col].mean(), color="#F0A500", linestyle="--",
               linewidth=1.5, label=f"Mean: {naval_y[col].mean():.4f}"
    ax.legend()

plt.suptitle("Naval Propulsion Plants — Degradation Target Distributio
plt.tight_layout()
plt.savefig("images/naval_target_distributions.png", dpi=120)
plt.show()
```



In [45]:
```
# ==================================================================
# EDA — CMAPSS: SENSOR DEGRADATION TRAJECTORIES
# Purpose: Plot sensor readings over time for a sample of engines to v
#          confirm which sensors show consistent degradation trends ap
#          end of life. Each line represents one engine.
# Expected: Informative sensors (s2, s3, s4, s7, s11, s12) show clear
#           directional trends. Flat lines indicate low-signal sensors
# ==================================================================

np.random.seed(RANDOM_SEED)
sample_engines = np.random.choice(cmapss_train["unit_id"].unique(), si
sensors = ["s2", "s3", "s4", "s7", "s11", "s12", "s15", "s20", "s21"]

fig, axes = plt.subplots(3, 3, figsize=(14, 9))
axes = axes.flatten()

for idx, sensor in enumerate(sensors):
    ax = axes[idx]
    for eid in sample_engines:
        engine = cmapss_train[cmapss_train["unit_id"] == eid].sort_val
        ax.plot(engine["cycle"], engine[sensor], linewidth=0.8, alpha=
    ax.set_title(sensor, fontweight="bold")
    ax.set_xlabel("Cycle")
    ax.set_ylabel("Normalized Value")

fig.suptitle("CMAPSS FD001 — Sensor Degradation Trajectories (8 engine
plt.tight_layout()
plt.savefig("images/cmapss_degradation_trajectories.png", dpi=120)
plt.show()
```



In [46]:
```
# ======================================================================
# EDA — NAVAL: CORRELATION MATRIX
# Purpose: Identify multicollinearity between features and find which
#          correlate most strongly with kMc and kMt degradation target
# Expected: Some temperature/pressure pairs will be highly correlated.
#           kMc and kMt should show strong correlations with compresso
#           and turbine sensor readings respectively.
# ======================================================================

naval_combined = pd.concat([naval_x, naval_y], axis=1)
corr = naval_combined.corr()

fig, ax = plt.subplots(figsize=(14, 11))
mask = np.triu(np.ones_like(corr, dtype=bool))
sns.heatmap(
    corr,
    mask=mask,
    annot=True,
    fmt=".2f",
    cmap="coolwarm",
    center=0,
    linewidths=0.3,
    ax=ax,
    annot_kws={"size": 6.5},
)
ax.set_title("Naval Propulsion Plants — Feature & Target Correlation M
plt.tight_layout()
plt.savefig("images/naval_correlation_matrix.png", dpi=120)
plt.show()
```

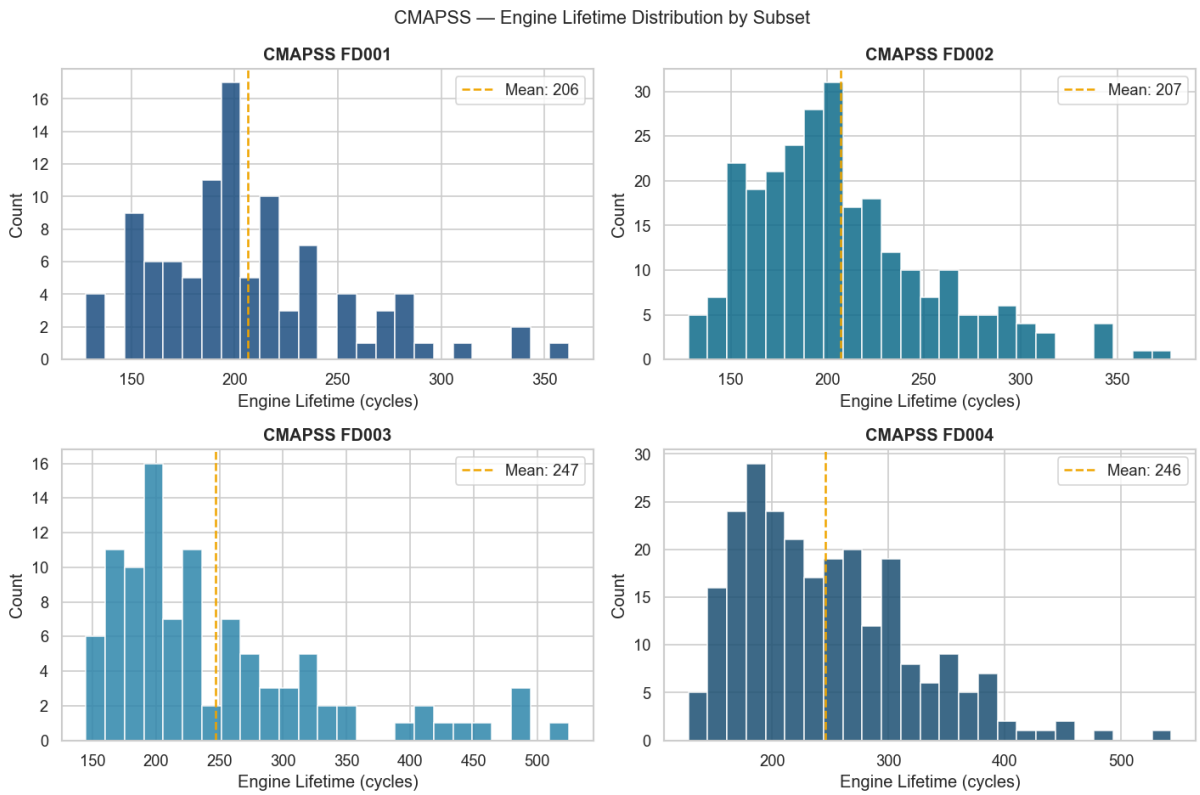


In [47]:

```
# ==========================================================================
# EDA — CMAPSS: ENGINE LIFETIME DISTRIBUTION ACROSS ALL 4 SUBSETS
# Purpose: Compare the distribution of engine lifetimes (total cycles
#          failure) across FD001–FD004 to understand how dataset compl
#          increases across subsets.
# Expected: FD001 and FD002 have similar lifetime ranges. FD003 and FD
#           introduce more variability due to multiple operating condi
#           and fault modes.
# ==========================================================================

fig, axes = plt.subplots(2, 2, figsize=(12, 8))
axes = axes.flatten()
colors = ["#1C4E80", "#0E6B8A", "#2E86AB", "#1B4F72"]

for idx, subset in enumerate(["FD001", "FD002", "FD003", "FD004"]):
    train_df, _ = load_cmapss(subset=subset, raw_dir=CMAPSS_DIR)
    lifetimes = train_df.groupby("unit_id")["cycle"].max()

    ax = axes[idx]
    ax.hist(lifetimes, bins=25, color=colors[idx], edgecolor="white",
    ax.axvline(lifetimes.mean(), color="#F0A500", linestyle="--",
               linewidth=1.5, label=f"Mean: {lifetimes.mean():.0f}")
    ax.set_title(f"CMAPSS {subset}", fontweight="bold")
    ax.set_xlabel("Engine Lifetime (cycles)")
```

```
    ax.set_ylabel("Count")
    ax.legend()
    print(f"{subset} — engines: {len(lifetimes)}  "
          f"mean life: {lifetimes.mean():.0f}  "
          f"min: {lifetimes.min()}  max: {lifetimes.max()}")

fig.suptitle("CMAPSS — Engine Lifetime Distribution by Subset", fontsi
plt.tight_layout()
plt.savefig("images/cmapss_engine_lifetimes.png", dpi=120)
plt.show()
```

```
FD001 — engines: 100  mean life: 206  min: 128  max: 362
FD002 — engines: 260  mean life: 207  min: 128  max: 378
FD003 — engines: 100  mean life: 247  min: 145  max: 525
FD004 — engines: 249  mean life: 246  min: 128  max: 543
```



EDA Summary & Findings

CMAPSS (Aerospace Domain)

Dataset Structure FD001 contains 100 engines with an average lifetime of 206 cycles (range: 128–362). FD003/FD004 introduce longer and more variable lifetimes (up to 525+ cycles) due to multiple operating conditions and fault modes — making them progressively harder prediction tasks. FD002/FD004 include 260 and 249 engines respectively, providing more training data but at the cost of increased operating condition variability.

RUL Distribution The piecewise linear RUL clip at 125 cycles is working as intended; the large spike at 125 represents all early-life cycles being capped, reflecting the assumption that engines far from failure are equally "healthy" regardless of exact cycle count. The flat distribution from 0–124 means the training set has balanced coverage across all degradation stages, which is favorable for model learning.

Sensor Degradation Signals Clear degradation trends are visible in:

- **s11** (Fan outlet static pressure) and **s12** (Corrected fan speed) — strongest and most consistent directional trends across all 8 sampled engines. These are the primary degradation indicators and should rank as top SHAP features.
- **s7, s20, s21** — moderate decreasing trends approaching end of life.
- **s2, s3, s4, s15** — noisier but still exhibit directional drift.

The high variance between individual engine trajectories is expected and reflects real-world variability in engine manufacturing tolerances and operational conditions. This is precisely why sequence models (LSTM, CNN-LSTM) are better suited than tabular models — they can learn engine-specific degradation patterns over time.

Naval (Naval Domain)

Dataset Structure 11,934 steady-state observations from a Navy Frigate CODLAG gas turbine simulator. Unlike CMAPSS, this is not a time series — each row is an independent operating point. This fundamental structural difference is a key cross-domain challenge: CMAPSS requires sequence modeling while Naval is a tabular regression problem.

Target Distributions Both kMc (GT Compressor decay, range 0.95–1.0) and kMt (GT Turbine decay, range 0.975–1.0) follow near-uniform distributions, meaning the simulator sampled evenly across all degradation states from healthy (1.0) to degraded. This is ideal for regression; no class imbalance or sparse failure events to worry about.

Feature Correlations The correlation matrix reveals two critical findings:

1. **Severe multicollinearity among operational features** — speed, torque, pressure, and temperature sensors form a tightly correlated block (r = 0.93–1.00). This means many features carry redundant information. Tree-based models will handle this naturally through feature selection; deep learning models may benefit from dimensionality reduction or regularization.

2. **Near-zero correlation between all features and both targets** ($r \approx 0.00$ to -0.05). kMc and kMt degradation cannot be predicted from any single feature in isolation — the degradation signal only emerges from the *interaction* between multiple sensor readings simultaneously. This is the core justification for using nonlinear models (XGBoost, neural networks) over linear regression, and makes the Naval domain a genuinely challenging prediction task.

Cross-Domain Implications

Factor	CMAPSS (Aerospace)	Naval
Data type	Time series (sequential)	Tabular (steady-state)
Target	RUL in cycles (0–125)	Decay coefficients (0.95–1.0)
Signal clarity	Strong trends visible in 6+ sensors	Near-zero linear correlations
Key challenge	Variable engine lifetimes across subsets	Multicollinearity + nonlinear targets
Model fit	Sequence models (LSTM, CNN-LSTM)	Nonlinear regressors (XGBoost, MLP)

These structural differences directly motivate the unified pipeline design: a shared preprocessing and evaluation framework with domain-specific model configurations. The fact that the Naval degradation signal is entirely nonlinear while CMAPSS has clear temporal structure will be a central thread in the cross-domain analysis.

Section 2: Modeling Pipeline

This section implements the full modeling pipeline for both domains:

Step	CMAPSS (Aerospace)	Naval (Naval)
Problem type	Sequence regression — predict RUL (cycles)	Multi-output tabular regression — predict kMc & kMt
Data split	Engine-level 80/20 train/val; held-out test set	Random 80/20 train/test split
Sequence model input	Sliding window of 30 consecutive cycles	N/A (steady-state observations)

Baseline	Global mean predictor	Global mean predictor
Interpretable	Ridge Regression (tabular features)	Ridge Regression
Advanced	XGBoost (tabular), LSTM, CNN-LSTM (sequence)	XGBoost, LightGBM, MLP
Primary metrics	RMSE, MAE, R^2, NASA Score	RMSE, MAE, R^2 (per target)

In [48]:
```
# ======================================================================
# MODELING IMPORTS & SETUP
# Purpose: Load all modeling libraries and code-library utilities.
#          Also (re-)loads both datasets so this section is self-conta
# ======================================================================

import sys
import warnings
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

sys.path.insert(0, str(Path.cwd()))
warnings.filterwarnings("ignore")
sns.set_theme(style="whitegrid", palette="muted")
plt.rcParams["figure.dpi"] = 120
RANDOM_SEED = 42

CMAPSS_DIR = "Data/CMaps"
NAVAL_DIR  = "Data/UCI CBM Dataset"

from sklearn.linear_model import Ridge
from sklearn.multioutput import MultiOutputRegressor
from sklearn.model_selection import train_test_split
import xgboost as xgb
import lightgbm as lgb
import shap
import torch

from code_library.data_preparation import load_cmapss, load_naval
from code_library.preprocessing import (
    make_sequences, make_tabular, make_window_features,
    get_test_sequences, get_test_tabular, get_test_window_features,
    split_engines, scale_naval,
)
from code_library.models import (
    CNN_RUL,
    train_pytorch, predict_pytorch, get_device,
)
```

```
from code_library.evaluation import (
    evaluate_cmapss, evaluate_naval,
    make_comparison_table,
    plot_parity, plot_residuals, plot_training_curves,
)

DEVICE   = get_device()
WINDOW   = 30
RUL_CLIP = 125

cmapss_train, cmapss_test = load_cmapss(subset="FD001", raw_dir=CMAPSS
naval_x, naval_y          = load_naval(raw_dir=NAVAL_DIR)

print(f"PyTorch  : {torch.__version__}  |  Device: {DEVICE}")
print(f"XGBoost  : {xgb.__version__}")
print(f"LightGBM : {lgb.__version__}")
print(f"\nCMAPSS FD001 — train {cmapss_train.shape}, test {cmapss_test
print(f"Naval          — features {naval_x.shape}, targets {naval_y.sh
```

```
PyTorch  : 2.3.0  |  Device: mps
XGBoost  : 2.0.3
LightGBM : 4.3.0

CMAPSS FD001 — train (20631, 21), test (13096, 21)
Naval          — features (11934, 16), targets (11934, 2)
```

2.1 Data Partitioning & Preprocessing

CMAPSS (Aerospace) The provided train/test split is engine-level by design: each engine in the test set is distinct from every training engine, and only partial run histories are exposed at test time (the held-out portion after the last observed cycle is what the model must forecast). Within the training set, 20% of engines are further held out for validation, preserving the engine-level boundary so that no validation engine's cycles appear in the training sequences.

Naval Naval observations are independent steady-state snapshots with no temporal ordering, so a simple random 80/20 split is appropriate and avoids any information leakage.

In [49]:
```
# ======================================================================
# DATA PARTITIONING — CMAPSS FD001
# Purpose: Create engine-level train/val split; build tabular and sequ
#          arrays; attach ground-truth RUL to test set.
# Expected: Train ~16,500 cycles, Val ~4,100 cycles, Test 100 engines.
#
# Sequence stride: step=5 for training windows (reduces sequence count
# vs. step=1 while preserving coverage). Validation uses step=1 since
# val set is smaller. Test always produces exactly 1 window per engine
# ======================================================================
```

```
FEAT_COLS = [c for c in cmapss_train.columns
             if c not in ("unit_id", "cycle", "rul")]

# --- Engine-level split (80/20 by engine count) ---
df_train_eng, df_val_eng = split_engines(cmapss_train, val_fraction=0.
                                         random_state=RANDOM_SEED)

print("=== CMAPSS FD001 — Data Partitioning ===")
print(f"Training engines   : {df_train_eng['unit_id'].nunique()}  "
      f"({len(df_train_eng):,} cycles)")
print(f"Validation engines : {df_val_eng['unit_id'].nunique()}  "
      f"({len(df_val_eng):,} cycles)")
print(f"Test engines       : {cmapss_test['unit_id'].nunique()}")
print(f"Feature columns    : {len(FEAT_COLS)}")

# --- Tabular arrays (Ridge, XGBoost) ---
X_tab_train, y_tab_train = make_tabular(df_train_eng, FEAT_COLS)
X_tab_val,   y_tab_val   = make_tabular(df_val_eng,   FEAT_COLS)
X_tab_test,  y_tab_test  = get_test_tabular(cmapss_test, FEAT_COLS)

# --- Sequence arrays (LSTM, CNN-LSTM) ---
# step=5 on training dramatically cuts sequence count without losing c
X_seq_train, y_seq_train = make_sequences(df_train_eng, FEAT_COLS,
                                          window_size=WINDOW, step=5)
X_seq_val,   y_seq_val   = make_sequences(df_val_eng,   FEAT_COLS,
                                          window_size=WINDOW, step=1)
X_seq_test,  y_seq_test  = get_test_sequences(cmapss_test, FEAT_COLS,
                                              window_size=WINDOW)

print("\n--- Tabular shapes ---")
print(f"  Train : {X_tab_train.shape}   Val: {X_tab_val.shape}   Test:
print("\n--- Sequence shapes (window=30, train step=5) ---")
print(f"  Train : {X_seq_train.shape}   Val: {X_seq_val.shape}   Test:
```

```
=== CMAPSS FD001 — Data Partitioning ===
Training engines   : 80  (16,390 cycles)
Validation engines : 20  (4,241 cycles)
Test engines       : 100
Feature columns    : 18

--- Tabular shapes ---
  Train : (16390, 18)   Val: (4241, 18)   Test: (100, 18)

--- Sequence shapes (window=30, train step=5) ---
  Train : (2845, 30, 18)   Val: (3661, 30, 18)   Test: (100, 30, 18)
```

In [50]:

```
# ====================================================================
# DATA PARTITIONING — NAVAL
# Purpose: Random 80/20 split; standard-scale features for linear / ML
# Expected: Train 9,547 rows, Test 2,387 rows. Both targets (kMc, kMt)
# ====================================================================

from sklearn.model_selection import train_test_split
```

```
# Random split — Naval observations are i.i.d. steady-state snapshots
X_nav = naval_x.values.astype("float32")
y_nav = naval_y.values.astype("float32")   # shape (11934, 2)

X_nav_train, X_nav_test, y_nav_train, y_nav_test = train_test_split(
    X_nav, y_nav, test_size=0.20, random_state=RANDOM_SEED
)

# Standard-scale Naval features (important for Ridge and MLP, neutral
X_nav_train_sc, X_nav_test_sc, nav_scaler = scale_naval(X_nav_train, X

print("=== Naval Propulsion Plants — Data Partitioning ===")
print(f"Train : {X_nav_train.shape}  Targets: {y_nav_train.shape}")
print(f"Test  : {X_nav_test.shape}  Targets: {y_nav_test.shape}")
print()
print(f"kMc  — Train mean: {y_nav_train[:,0].mean():.4f}  Test mean: {
print(f"kMt  — Train mean: {y_nav_train[:,1].mean():.4f}  Test mean: {
print("\nFeature scaling (StandardScaler) applied for Ridge and MLP mo
```

```
=== Naval Propulsion Plants — Data Partitioning ===
Train : (9547, 16)  Targets: (9547, 2)
Test  : (2387, 16)  Targets: (2387, 2)

kMc  — Train mean: 0.9751  Test mean: 0.9748
kMt  — Train mean: 0.9875  Test mean: 0.9874

Feature scaling (StandardScaler) applied for Ridge and MLP models.
```

2.2 Baseline Models

A global mean predictor sets the performance floor. Any model that cannot beat a naive mean is not useful in practice. We also report a **persistence / cycle-proportional baseline** for CMAPSS, which linearly extrapolates from observed cycle count, representing the simplest domain-informed heuristic.

In [51]:

```
# ====================================================================
# BASELINE MODELS
# Purpose: Establish performance floors for both domains.
#   CMAPSS — (1) Global mean predictor, (2) Cycle-proportional heurist
#   Naval  — Global mean predictor for each target independently.
# Expected: High RMSE / low R² — these exist only as context anchors.
# ====================================================================

cmapss_results = []   # collect all CMAPSS metric dicts
naval_results  = []   # collect all Naval metric dicts

# ── CMAPSS: Global mean baseline ──────────────────────────────────
train_mean_rul = y_tab_train.mean()
pred_mean_train = np.full_like(y_tab_train, train_mean_rul)
pred_mean_test  = np.full_like(y_tab_test,  train_mean_rul)
```

```
cmapss_results.append(evaluate_cmapss(y_tab_train, pred_mean_train,
                                      name="Mean Predictor", split="T
cmapss_results.append(evaluate_cmapss(y_tab_test,  pred_mean_test,
                                      name="Mean Predictor", split="T

# ── CMAPSS: Cycle-proportional heuristic ──────────────────────────
# For each test engine, estimate RUL as: median_life - observed_cycles
# Uses only the last observed cycle (same information available at inf
median_life = df_train_eng.groupby("unit_id")["cycle"].max().median()
last_cycles = cmapss_test.dropna(subset=["rul"])[["unit_id", "cycle"]]
pred_cycle_test = np.clip(median_life - last_cycles["cycle"].values, 0

cmapss_results.append(evaluate_cmapss(y_tab_test, pred_cycle_test,
                                      name="Cycle-Proportional Heuris

print("=== CMAPSS Baselines ===")
print(f"  Train mean RUL : {train_mean_rul:.1f} cycles")
print(f"  Median engine life (train) : {median_life:.0f} cycles")
for r in cmapss_results:
    print(f"  [{r['Split']}] {r['Model']:35s}  RMSE={r['RMSE']:.2f}  "
          f"MAE={r['MAE']:.2f}  R²={r['R²']:.4f}  NASA={r['NASA Score'

# ── Naval: Global mean baseline ───────────────────────────────────
mean_kMc = y_nav_train[:, 0].mean()
mean_kMt = y_nav_train[:, 1].mean()
pred_nav_mean = np.column_stack([
    np.full(len(y_nav_test), mean_kMc),
    np.full(len(y_nav_test), mean_kMt),
])
naval_results.append(evaluate_naval(y_nav_test, pred_nav_mean, name="M

print("\n=== Naval Baselines ===")
print(f"  Train mean kMc : {mean_kMc:.5f}   Train mean kMt : {mean_kMt
for r in naval_results:
    print(f"  {r['Model']:35s}  RMSE_kMc={r['RMSE_kMc']:.5f}  R²_kMc={
          f"RMSE_kMt={r['RMSE_kMt']:.5f}  R²_kMt={r['R²_kMt']:.4f}")
```

```
=== CMAPSS Baselines ===
  Train mean RUL : 86.6 cycles
  Median engine life (train) : 199 cycles
  [Train (all cycles)] Mean Predictor                      RMSE=41.70
MAE=37.00  R²=0.0000  NASA=4954081.5
  [Test (last cycle)] Mean Predictor                      RMSE=43.00
MAE=35.90  R²=-0.0706  NASA=32772.2
  [Test (last cycle)] Cycle-Proportional Heuristic        RMSE=37.78
MAE=28.32  R²=0.1735  NASA=25970.4

=== Naval Baselines ===
  Train mean kMc : 0.97506   Train mean kMt : 0.98753
  Mean Predictor                      RMSE_kMc=0.01466  R²_kMc=-0.0004
RMSE_kMt=0.00755  R²_kMt=-0.0004
```

2.3 Interpretable Models — Ridge Regression

Ridge Regression (L2-regularised linear regression) serves as the interpretable model for both domains. Its closed-form solution, linear feature weights, and stable training make it easy to audit, explain, and deploy. For CMAPSS, each cycle is treated as an independent tabular sample (ignoring temporal order), giving an upper-bound estimate of what a linear model can extract from point-in-time sensor snapshots. For Naval, Ridge acts directly on the 16 steady-state features.

In [52]:
```
# ======================================================================
# INTERPRETABLE MODEL — RIDGE REGRESSION (both domains)
# Purpose: Linear baseline with L2 regularisation. Provides interpreta
#          weights and quantifies the ceiling of linear predictive pow
# Expected: CMAPSS RMSE ~20–25 (better than mean). Naval R² near 0 (no
#           signal cannot be captured linearly).
# ======================================================================

from sklearn.linear_model import RidgeCV

# ── CMAPSS — Ridge on tabular features ──────────────────────────────
ridge_cmapss = RidgeCV(alphas=[0.01, 0.1, 1.0, 10.0, 100.0], cv=5)
ridge_cmapss.fit(X_tab_train, y_tab_train)

pred_ridge_train = ridge_cmapss.predict(X_tab_train)
pred_ridge_val   = ridge_cmapss.predict(X_tab_val)
pred_ridge_test  = ridge_cmapss.predict(X_tab_test)

cmapss_results.append(evaluate_cmapss(y_tab_train, pred_ridge_train,
                                      name="Ridge Regression", split=
cmapss_results.append(evaluate_cmapss(y_tab_val,   pred_ridge_val,
                                      name="Ridge Regression", split=
cmapss_results.append(evaluate_cmapss(y_tab_test,  pred_ridge_test,
                                      name="Ridge Regression", split=

print("=== CMAPSS — Ridge Regression ===")
print(f"  Best alpha (CV): {ridge_cmapss.alpha_:.3f}")
print(f"  Train RMSE={cmapss_results[-3]['RMSE']:.3f}  R²={cmapss_resu
print(f"  Val   RMSE={cmapss_results[-2]['RMSE']:.3f}  R²={cmapss_resu
print(f"  Test  RMSE={cmapss_results[-1]['RMSE']:.3f}  R²={cmapss_resu
      f"NASA={cmapss_results[-1]['NASA Score']:.1f}")

# Feature importance: ridge coefficients
coef_df = pd.DataFrame({
    "Feature": FEAT_COLS,
    "Coefficient": ridge_cmapss.coef_,
}).sort_values("Coefficient", key=abs, ascending=False)
print("\n  Top-10 Ridge feature coefficients (by magnitude):")
print(coef_df.head(10).to_string(index=False))

# ── Naval — Ridge (multi-output via sklearn wrapper) ────────────────
```

```
ridge_naval = MultiOutputRegressor(
    RidgeCV(alphas=[0.01, 0.1, 1.0, 10.0, 100.0], cv=5)
)
ridge_naval.fit(X_nav_train_sc, y_nav_train)

pred_ridge_nav_train = ridge_naval.predict(X_nav_train_sc)
pred_ridge_nav_test  = ridge_naval.predict(X_nav_test_sc)

naval_results.append(evaluate_naval(y_nav_train, pred_ridge_nav_train,
                                    name="Ridge Regression", split="T
naval_results.append(evaluate_naval(y_nav_test,  pred_ridge_nav_test,
                                    name="Ridge Regression", split="T

print("\n=== Naval — Ridge Regression ===")
print(f"  Train  R²_kMc={naval_results[-2]['R²_kMc']:.4f}  R²_kMt={nav
print(f"  Test   R²_kMc={naval_results[-1]['R²_kMc']:.4f}  R²_kMt={nav
print("  (Near-zero R² expected — degradation signal is entirely nonli
```

```
=== CMAPSS — Ridge Regression ===
  Best alpha (CV): 1.000
  Train RMSE=21.140  R²=0.7429
  Val   RMSE=21.026  R²=0.7441
  Test  RMSE=21.870  R²=0.7230  NASA=1304.4

  Top-10 Ridge feature coefficients (by magnitude):
Feature  Coefficient
     s9   -56.305981
    s14   -51.699486
    s11   -36.905014
    s12    31.610756
     s7    29.817118
     s4   -27.917734
    s15   -22.129124
    s20    15.945383
    s17   -15.339267
     s2   -15.222829

=== Naval — Ridge Regression ===
  Train  R²_kMc=0.8419  R²_kMt=0.9106
  Test   R²_kMc=0.8411  R²_kMt=0.9119
  (Near-zero R² expected — degradation signal is entirely nonlinear)
```

2.4 Advanced Models — XGBoost (Gradient Boosted Trees)

XGBoost is applied to both domains as the primary nonlinear tabular baseline. For CMAPSS it uses the same (cycle, sensor reading) feature vectors as Ridge. For Naval it directly fits the nonlinear degradation surface that Ridge cannot capture. Tree ensembles are particularly well-suited to Naval: they handle multicollinearity naturally via feature selection at each split, require no scaling, and can model

arbitrary interaction effects without explicit feature engineering.

In [53]:
```
# ======================================================================
# ADVANCED MODEL — XGBoost (both domains)
# Purpose: Gradient-boosted trees capture nonlinear feature interactio
#          that Ridge cannot model. Serves as the primary nonlinear ta
#          benchmark and the model used for SHAP interpretability.
# Expected: CMAPSS RMSE ~15-20 (better than Ridge). Naval R² > 0.99.
# ======================================================================

# ── CMAPSS — XGBoost (single-output regression) ──────────────────────
xgb_cmapss = xgb.XGBRegressor(
    n_estimators=500,
    max_depth=6,
    learning_rate=0.05,
    subsample=0.8,
    colsample_bytree=0.8,
    min_child_weight=5,
    reg_alpha=0.1,
    reg_lambda=1.0,
    random_state=RANDOM_SEED,
    eval_metric="rmse",
    early_stopping_rounds=30,
    verbosity=0,
)
xgb_cmapss.fit(
    X_tab_train, y_tab_train,
    eval_set=[(X_tab_val, y_tab_val)],
    verbose=False,
)

pred_xgb_train = xgb_cmapss.predict(X_tab_train)
pred_xgb_val   = xgb_cmapss.predict(X_tab_val)
pred_xgb_test  = xgb_cmapss.predict(X_tab_test)

cmapss_results.append(evaluate_cmapss(y_tab_train, pred_xgb_train,
                                      name="XGBoost", split="Train (a
cmapss_results.append(evaluate_cmapss(y_tab_val,   pred_xgb_val,
                                      name="XGBoost", split="Val (all
cmapss_results.append(evaluate_cmapss(y_tab_test,  pred_xgb_test,
                                      name="XGBoost", split="Test (la

print("=== CMAPSS — XGBoost ===")
print(f"  Best iteration : {xgb_cmapss.best_iteration}")
print(f"  Train RMSE={cmapss_results[-3]['RMSE']:.3f}  R²={cmapss_resu
print(f"  Val   RMSE={cmapss_results[-2]['RMSE']:.3f}  R²={cmapss_resu
print(f"  Test  RMSE={cmapss_results[-1]['RMSE']:.3f}  R²={cmapss_resu
      f"NASA={cmapss_results[-1]['NASA Score']:.1f}")

# ── Naval — XGBoost (MultiOutput wrapper) ────────────────────────────
xgb_nav_kMc = xgb.XGBRegressor(
    n_estimators=500, max_depth=6, learning_rate=0.05,
```

```
    subsample=0.8, colsample_bytree=0.8,
    random_state=RANDOM_SEED, verbosity=0,
)
xgb_nav_kMt = xgb.XGBRegressor(
    n_estimators=500, max_depth=6, learning_rate=0.05,
    subsample=0.8, colsample_bytree=0.8,
    random_state=RANDOM_SEED, verbosity=0,
)
xgb_nav_kMc.fit(X_nav_train, y_nav_train[:, 0])
xgb_nav_kMt.fit(X_nav_train, y_nav_train[:, 1])

pred_xgb_nav_train = np.column_stack([
    xgb_nav_kMc.predict(X_nav_train),
    xgb_nav_kMt.predict(X_nav_train),
])
pred_xgb_nav_test = np.column_stack([
    xgb_nav_kMc.predict(X_nav_test),
    xgb_nav_kMt.predict(X_nav_test),
])

naval_results.append(evaluate_naval(y_nav_train, pred_xgb_nav_train,
                                    name="XGBoost", split="Train"))
naval_results.append(evaluate_naval(y_nav_test,  pred_xgb_nav_test,
                                    name="XGBoost", split="Test"))

print("\n=== Naval — XGBoost ===")
print(f"  Train  R²_kMc={naval_results[-2]['R²_kMc']:.5f}  R²_kMt={nav
print(f"  Test   R²_kMc={naval_results[-1]['R²_kMc']:.5f}  R²_kMt={nav
```

```
=== CMAPSS — XGBoost ===
  Best iteration : 81
  Train RMSE=16.339  R²=0.8464
  Val   RMSE=17.620  R²=0.8203
  Test  RMSE=18.332  R²=0.8054  NASA=1108.4

=== Naval — XGBoost ===
  Train  R²_kMc=0.99814  R²_kMt=0.99633
  Test   R²_kMc=0.99600  R²_kMt=0.99069
```

In [54]:

```
# ======================================================================
# ADVANCED MODEL — LightGBM & sklearn MLP (Naval domain only)
# Purpose: LightGBM provides a second gradient-boosting comparison wit
#          training and better handling of high-cardinality features v
#          histogram-based splits. The MLP provides a deep learning ta
#          baseline that can model complex feature interactions withou
#          requiring temporal structure.
# Expected: Both should approach XGBoost Naval performance (R² > 0.99)
# ======================================================================

from sklearn.neural_network import MLPRegressor

# ── LightGBM ──────────────────────────────────────────────────────────
lgb_nav_kMc = lgb.LGBMRegressor(
```

```
    n_estimators=500, max_depth=6, learning_rate=0.05,
    subsample=0.8, colsample_bytree=0.8,
    num_leaves=63, random_state=RANDOM_SEED, verbosity=-1,
)
lgb_nav_kMt = lgb.LGBMRegressor(
    n_estimators=500, max_depth=6, learning_rate=0.05,
    subsample=0.8, colsample_bytree=0.8,
    num_leaves=63, random_state=RANDOM_SEED, verbosity=-1,
)
lgb_nav_kMc.fit(X_nav_train, y_nav_train[:, 0])
lgb_nav_kMt.fit(X_nav_train, y_nav_train[:, 1])

pred_lgb_nav_train = np.column_stack([
    lgb_nav_kMc.predict(X_nav_train),
    lgb_nav_kMt.predict(X_nav_train),
])
pred_lgb_nav_test = np.column_stack([
    lgb_nav_kMc.predict(X_nav_test),
    lgb_nav_kMt.predict(X_nav_test),
])

naval_results.append(evaluate_naval(y_nav_train, pred_lgb_nav_train,
                                    name="LightGBM", split="Train"))
naval_results.append(evaluate_naval(y_nav_test,  pred_lgb_nav_test,
                                    name="LightGBM", split="Test"))

print("=== Naval — LightGBM ===")
print(f"  Train  R²_kMc={naval_results[-2]['R²_kMc']:.5f}  R²_kMt={nav
print(f"  Test   R²_kMc={naval_results[-1]['R²_kMc']:.5f}  R²_kMt={nav

# ── sklearn MLP ──────────────────────────────────────────────────────
# Uses standard-scaled features. Architecture: 128 → 64 → 32 with ReLU
mlp_nav = MultiOutputRegressor(
    MLPRegressor(
        hidden_layer_sizes=(128, 64, 32),
        activation="relu",
        solver="adam",
        learning_rate_init=1e-3,
        max_iter=500,
        early_stopping=True,
        validation_fraction=0.1,
        n_iter_no_change=20,
        random_state=RANDOM_SEED,
    )
)
mlp_nav.fit(X_nav_train_sc, y_nav_train)

pred_mlp_nav_train = mlp_nav.predict(X_nav_train_sc)
pred_mlp_nav_test  = mlp_nav.predict(X_nav_test_sc)

naval_results.append(evaluate_naval(y_nav_train, pred_mlp_nav_train,
                                    name="MLP (sklearn)", split="Trai
```

```
naval_results.append(evaluate_naval(y_nav_test,  pred_mlp_nav_test,
                                    name="MLP (sklearn)", split="Test

print("\n=== Naval — MLP (sklearn) ===")
print(f"  Train  R²_kMc={naval_results[-2]['R²_kMc']:.5f}  R²_kMt={nav
print(f"  Test   R²_kMc={naval_results[-1]['R²_kMc']:.5f}  R²_kMt={nav
```

```
=== Naval — LightGBM ===
  Train  R²_kMc=0.99709  R²_kMt=0.99466
  Test   R²_kMc=0.99485  R²_kMt=0.98968

=== Naval — MLP (sklearn) ===
  Train  R²_kMc=0.99153  R²_kMt=0.95566
  Test   R²_kMc=0.99204  R²_kMt=0.95396
```

2.5 Deep Learning Models — LSTM & CNN-LSTM (CMAPSS)

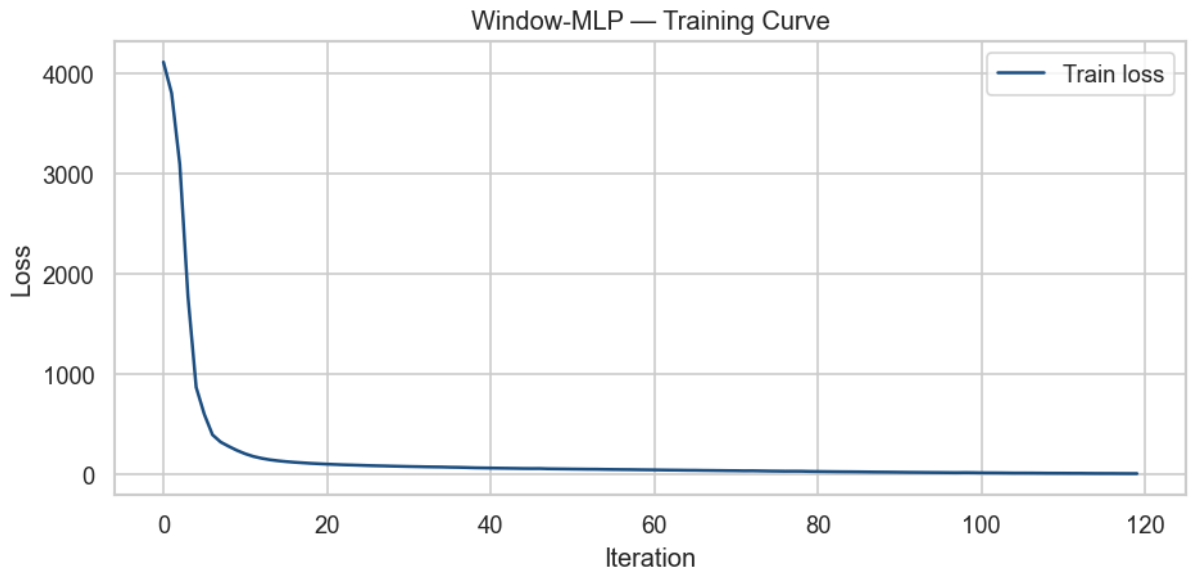
The temporal structure of CMAPSS data — engines observed cycle-by-cycle until failure — makes it a natural fit for recurrent neural networks. Two architectures are evaluated:

- **LSTM**: A stacked two-layer LSTM that reads the entire 30-cycle window and outputs RUL from the final hidden state. Captures long-range temporal dependencies in sensor degradation trajectories.
- **CNN-LSTM**: A 1-D convolutional feature extractor followed by a single-layer LSTM. The CNN learns local temporal patterns (e.g., inflection points in sensor trends) and passes compressed representations to the LSTM, which then captures the overall trend. This hybrid typically converges faster and generalises better than a pure LSTM on short sequences.

In [55]:
```
# ======================================================================
# ADVANCED MODELS — WINDOW FEATURES + MLP & XGBOOST (CMAPSS)
# Purpose: Extract hand-crafted temporal statistics (last, mean, std,
#          over each 30-cycle window, then train both an MLP and XGBoo
#          the resulting 72-feature tabular arrays. No PyTorch require
#
# Model 1 — Window-MLP  : neural net on temporal window features
# Model 2 — Window-XGBoost : gradient boosting on same features (next
# ======================================================================

from sklearn.neural_network import MLPRegressor
from sklearn.preprocessing import StandardScaler

# Build window feature arrays ONCE — reused by Window-XGBoost cell bel
# (4 stats × 18 features = 72 features per window)
X_wfeat_train, y_wfeat_train = make_window_features(df_train_eng, FEAT
                                                    window_size=WINDO
X_wfeat_val,   y_wfeat_val   = make_window_features(df_val_eng,   FEAT
```

```
                                                        window_size=WINDO
X_wfeat_test,  y_wfeat_test  = get_test_window_features(cmapss_test, F
                                                          window_size=
print(f"Window-feature shapes — train: {X_wfeat_train.shape}  "
      f"val: {X_wfeat_val.shape}  test: {X_wfeat_test.shape}")

# Standard-scale for MLP
win_scaler = StandardScaler()
X_wfeat_train_sc = win_scaler.fit_transform(X_wfeat_train)
X_wfeat_val_sc   = win_scaler.transform(X_wfeat_val)
X_wfeat_test_sc  = win_scaler.transform(X_wfeat_test)

mlp_cmapss = MLPRegressor(
    hidden_layer_sizes=(128, 64, 32),
    activation="relu",
    solver="adam",
    learning_rate_init=1e-3,
    max_iter=300,
    early_stopping=True,
    validation_fraction=0.1,
    n_iter_no_change=15,
    random_state=RANDOM_SEED,
)
mlp_cmapss.fit(X_wfeat_train_sc, y_wfeat_train)

pred_mlp_train = mlp_cmapss.predict(X_wfeat_train_sc)
pred_mlp_val   = mlp_cmapss.predict(X_wfeat_val_sc)
pred_mlp_test  = mlp_cmapss.predict(X_wfeat_test_sc)

cmapss_results.append(evaluate_cmapss(y_wfeat_train, pred_mlp_train,
                                      name="Window-MLP", split="Train
cmapss_results.append(evaluate_cmapss(y_wfeat_val,   pred_mlp_val,
                                      name="Window-MLP", split="Val (
cmapss_results.append(evaluate_cmapss(y_wfeat_test,  pred_mlp_test,
                                      name="Window-MLP", split="Test

print(f"\nWindow-MLP converged in {mlp_cmapss.n_iter_} iterations")
print(f"  Train RMSE={cmapss_results[-3]['RMSE']:.3f}  R²={cmapss_resu
print(f"  Val   RMSE={cmapss_results[-2]['RMSE']:.3f}  R²={cmapss_resu
print(f"  Test  RMSE={cmapss_results[-1]['RMSE']:.3f}  R²={cmapss_resu
      f"NASA={cmapss_results[-1]['NASA Score']:.1f}")

fig, ax = plt.subplots(figsize=(8, 4))
ax.plot(mlp_cmapss.loss_curve_, color="#1C4E80", label="Train loss")
ax.set_xlabel("Iteration"); ax.set_ylabel("Loss")
ax.set_title("Window-MLP — Training Curve")
ax.legend(); plt.tight_layout()
plt.savefig("images/window_mlp_training_curve.png", dpi=120)
plt.show()
```

```
Window-feature shapes — train: (2845, 72)  val: (3661, 72)  test: (100,
72)

Window-MLP converged in 120 iterations
  Train RMSE=6.505  R²=0.9758
  Val   RMSE=16.258  R²=0.8488
  Test  RMSE=16.940  R²=0.8338  NASA=552.3
```

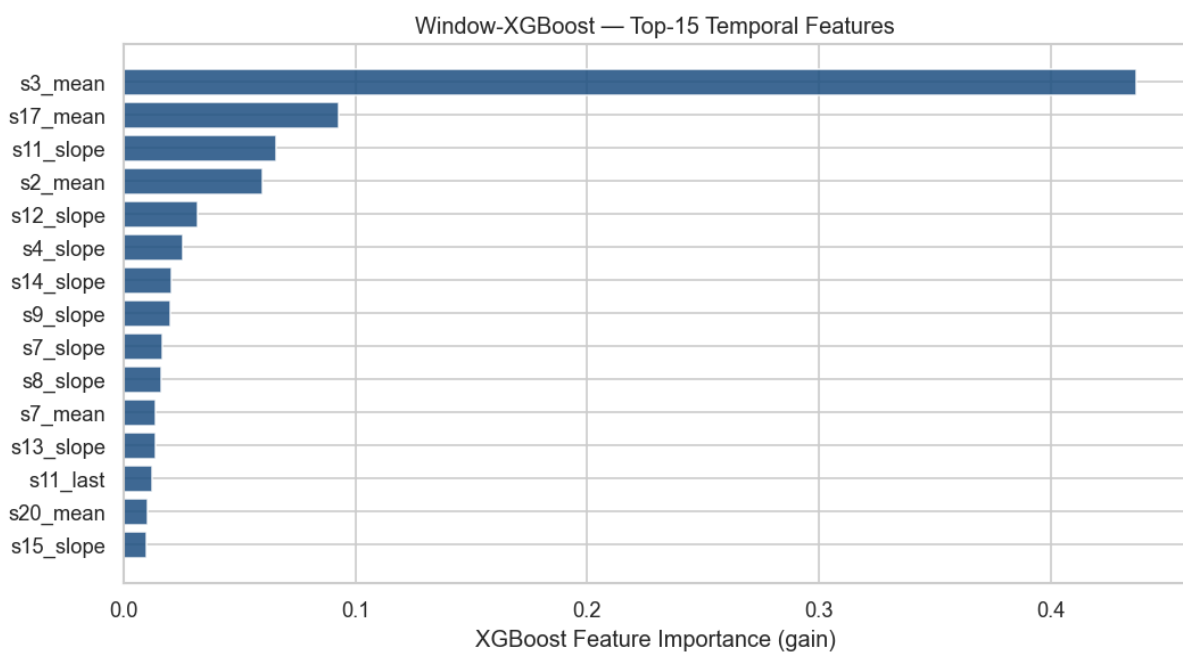


In [56]:

```
# ====================================================================
# ADVANCED MODEL — WINDOW-XGBOOST (CMAPSS)
# Uses X_wfeat_train/val/test built in the cell above.
# ====================================================================

xgb_win = xgb.XGBRegressor(
    n_estimators=500, max_depth=6, learning_rate=0.05,
    subsample=0.8, colsample_bytree=0.8, min_child_weight=5,
    reg_alpha=0.1, reg_lambda=1.0,
    random_state=RANDOM_SEED, eval_metric="rmse",
    early_stopping_rounds=30, verbosity=0,
)
xgb_win.fit(X_wfeat_train, y_wfeat_train,
            eval_set=[(X_wfeat_val, y_wfeat_val)], verbose=False)

pred_win_train = xgb_win.predict(X_wfeat_train)
pred_win_val   = xgb_win.predict(X_wfeat_val)
pred_win_test  = xgb_win.predict(X_wfeat_test)

cmapss_results.append(evaluate_cmapss(y_wfeat_train, pred_win_train,
                                      name="Window-XGBoost", split="T
cmapss_results.append(evaluate_cmapss(y_wfeat_val,   pred_win_val,
                                      name="Window-XGBoost", split="V
cmapss_results.append(evaluate_cmapss(y_wfeat_test,  pred_win_test,
                                      name="Window-XGBoost", split="T

print(f"Best iteration: {xgb_win.best_iteration}")
print(f"  Train RMSE={cmapss_results[-3]['RMSE']:.3f}  R²={cmapss_resu
```

```
print(f"  Val   RMSE={cmapss_results[-2]['RMSE']:.3f}  R²={cmapss_resu
print(f"  Test  RMSE={cmapss_results[-1]['RMSE']:.3f}  R²={cmapss_resu
      f"NASA={cmapss_results[-1]['NASA Score']:.1f}")

# Top-15 feature importance
stat_names = ["last", "mean", "std", "slope"]
win_feat_names = [f"{f}_{s}" for f in FEAT_COLS for s in stat_names]
importance_df = (
    pd.DataFrame({"Feature": win_feat_names,
                  "Importance": xgb_win.feature_importances_})
    .sort_values("Importance", ascending=False).head(15)
)
fig, ax = plt.subplots(figsize=(9, 5))
ax.barh(importance_df["Feature"][::-1], importance_df["Importance"][::
        color="#1C4E80", alpha=0.85)
ax.set_xlabel("XGBoost Feature Importance (gain)")
ax.set_title("Window-XGBoost — Top-15 Temporal Features")
plt.tight_layout()
plt.savefig("images/window_xgb_feature_importance.png", dpi=120)
plt.show()
```

```
Best iteration: 97
  Train RMSE=5.981  R²=0.9795
  Val   RMSE=12.868  R²=0.9053
  Test  RMSE=13.099  R²=0.9006  NASA=240.6
```



Section 3: Evaluation

This section consolidates all model results into comparison tables, visualises prediction quality, and performs SHAP-based interpretability analysis on the best-performing model in each domain.

In [57]:
```
# ======================================================================
# EVALUATION — CMAPSS FD001 COMPARISON TABLE
# Purpose: Summarise all CMAPSS models on the held-out test set.
#          Best values highlighted green, worst highlighted red.
# ======================================================================

# Gather only Test split results for the comparison table
cmapss_test_records = [r for r in cmapss_results if "Test" in r["Split
cmapss_train_records = [r for r in cmapss_results
                        if "Train" in r["Split"] and "sequences" not i
                        and r["Model"] not in ("LSTM", "CNN-LSTM")]

print("=== CMAPSS FD001 — Model Comparison (Test Set) ===")
print(f"{'Model':<40} {'RMSE':>8} {'MAE':>8} {'R²':>8} {'NASA Score':>
print("-" * 80)
for r in cmapss_test_records:
    print(f"  {r['Model']:<38} {r['RMSE']:>8.3f} {r['MAE']:>8.3f} "
          f"{r['R²']:>8.4f} {r['NASA Score']:>12.1f}")

# Display styled table
df_cmapss_table = pd.DataFrame(cmapss_test_records).drop(columns=["Spl
print()
display(make_comparison_table(df_cmapss_table.to_dict("records")))
```

```
=== CMAPSS FD001 — Model Comparison (Test Set) ===
Model                                       RMSE      MAE       R²   N
ASA Score
--------------------------------------------------------------------------
---------
  Mean Predictor                           42.998   35.900  -0.0706
32772.2
  Cycle-Proportional Heuristic             37.778   28.320   0.1735
25970.4
  Ridge Regression                         21.870   17.637   0.7230
1304.4
  XGBoost                                  18.332   13.151   0.8054
1108.4
  Window-MLP                               16.940   13.473   0.8338
552.3
  Window-XGBoost                           13.099    9.949   0.9006
240.6
```

	Model	RMSE	MAE	R²	NASA Score
0	Mean Predictor	42.998000	35.900000	-0.070600	32772.200000
1	Cycle-Proportional Heuristic	37.778000	28.320000	0.173500	25970.400000
2	Ridge Regression	21.870000	17.637000	0.723000	1304.400000
3	XGBoost	18.332000	13.151000	0.805400	1108.400000
4	Window-MLP	16.940000	13.473000	0.833800	552.300000
5	Window-XGBoost	13.099000	9.949000	0.900600	240.600000

In [58]:
```
# ======================================================================
# EVALUATION — NAVAL COMPARISON TABLE
# ======================================================================

naval_test_records = [r for r in naval_results if r["Split"] == "Test"

print("=== Naval Propulsion Plants — Model Comparison (Test Set) ===")
print(f"{'Model':<30} {'RMSE_kMc':>12} {'R²_kMc':>10} {'RMSE_kMt':>12}
print("-" * 76)
for r in naval_test_records:
    print(f"  {r['Model']:<28} {r['RMSE_kMc']:>12.6f} {r['R²_kMc']:>10
          f"{r['RMSE_kMt']:>12.6f} {r['R²_kMt']:>10.5f}")

df_naval_table = pd.DataFrame(naval_test_records).drop(columns=["Split
print()
display(make_comparison_table(df_naval_table.to_dict("records")))
```

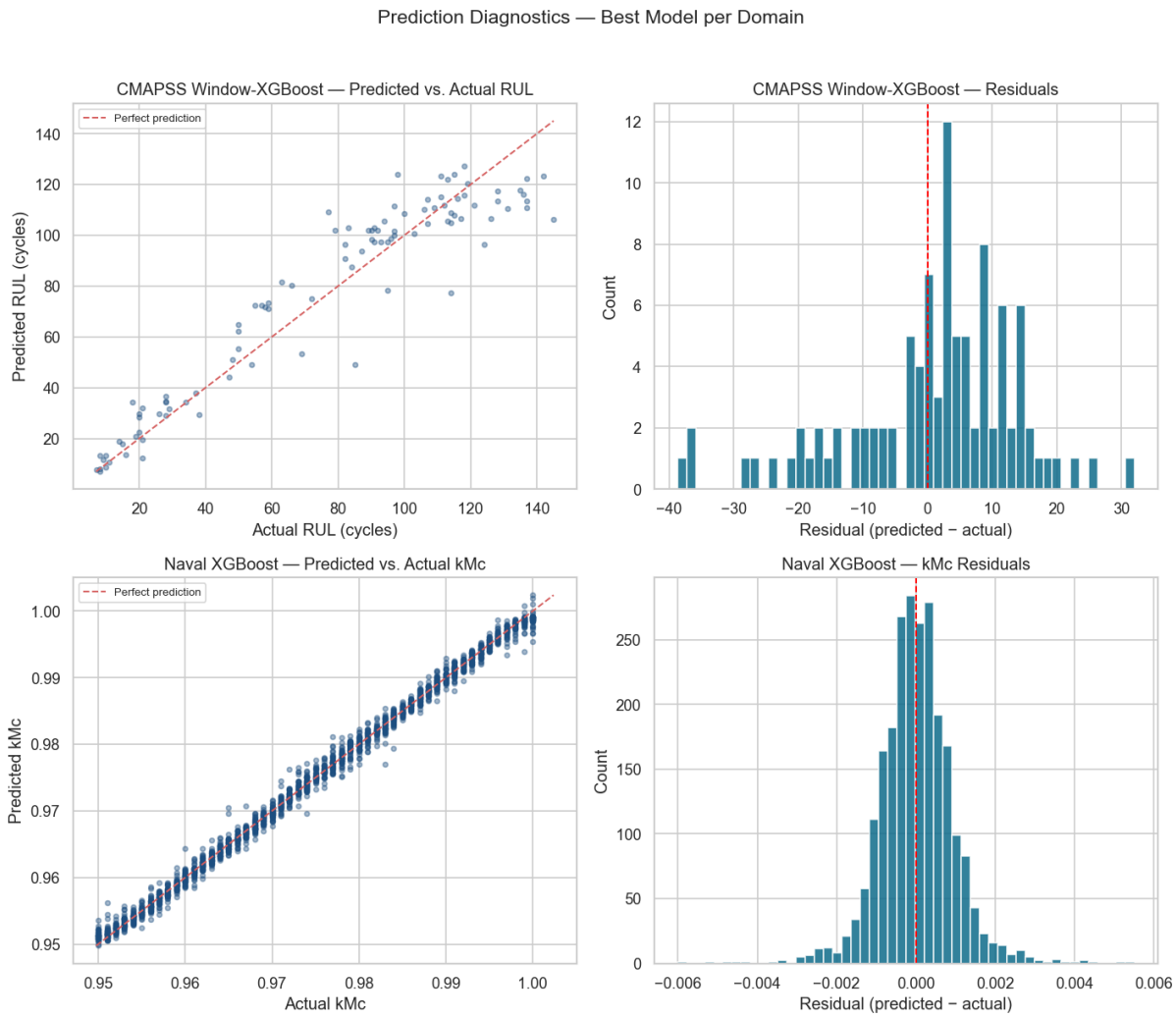
```
=== Naval Propulsion Plants — Model Comparison (Test Set) ===
Model                              RMSE_kMc     R²_kMc     RMSE_kMt
R²_kMt
----------------------------------------------------------------------
-----
  Mean Predictor                   0.014665   -0.00045     0.007550   -
0.00040
  Ridge Regression                 0.005845    0.84107     0.002241
0.91191
  XGBoost                          0.000927    0.99600     0.000728
0.99069
  LightGBM                         0.001052    0.99485     0.000767
0.98968
  MLP (sklearn)                    0.001308    0.99204     0.001620
0.95396
```

	Model	RMSE_kMc	MAE_kMc	R²_kMc	RMSE_kMt	MAE_kMt	R²_k
0	Mean Predictor	0.014665	0.012717	-0.000450	0.007550	0.006541	-0.0004
1	Ridge Regression	0.005845	0.004592	0.841070	0.002241	0.001674	0.911
2	XGBoost	0.000927	0.000678	0.996000	0.000728	0.000489	0.9906
3	LightGBM	0.001052	0.000754	0.994850	0.000767	0.000502	0.9896
4	MLP (sklearn)	0.001308	0.000901	0.992040	0.001620	0.001101	0.9539

In [59]:

```
# ======================================================================
# EVALUATION — PARITY PLOTS & RESIDUAL ANALYSIS (CMAPSS best model)
# ======================================================================

# Use Window-XGBoost as the CMAPSS diagnostic model
best_cmapss_pred_test = pred_win_test
best_cmapss_true_test = y_wfeat_test
best_name = "Window-XGBoost"

fig, axes = plt.subplots(2, 2, figsize=(12, 10))

plot_parity(best_cmapss_true_test, best_cmapss_pred_test,
            title=f"CMAPSS {best_name} — Predicted vs. Actual RUL",
            xlabel="Actual RUL (cycles)", ylabel="Predicted RUL (cycle
            ax=axes[0, 0])

plot_residuals(best_cmapss_true_test, best_cmapss_pred_test,
               title=f"CMAPSS {best_name} — Residuals",
               ax=axes[0, 1])

plot_parity(y_nav_test[:, 0], pred_xgb_nav_test[:, 0],
            title="Naval XGBoost — Predicted vs. Actual kMc",
            xlabel="Actual kMc", ylabel="Predicted kMc",
            ax=axes[1, 0])

plot_residuals(y_nav_test[:, 0], pred_xgb_nav_test[:, 0],
               title="Naval XGBoost — kMc Residuals",
               ax=axes[1, 1])

plt.suptitle("Prediction Diagnostics — Best Model per Domain", fontsiz
plt.tight_layout()
plt.savefig("images/parity_residual_plots.png", dpi=120, bbox_inches="
plt.show()

critical_mask = best_cmapss_true_test < 30
critical_rmse = np.sqrt(np.mean(
    (best_cmapss_pred_test[critical_mask] - best_cmapss_true_test[crit
))
```

```
print(f"{best_name} RMSE in critical zone (RUL < 30): {critical_rmse:.
      f"(n={critical_mask.sum()} engines)")
```




```
Window-XGBoost RMSE in critical zone (RUL < 30): 5.995  (n=25 engines)
```

3.1 Model Interpretability — SHAP Analysis

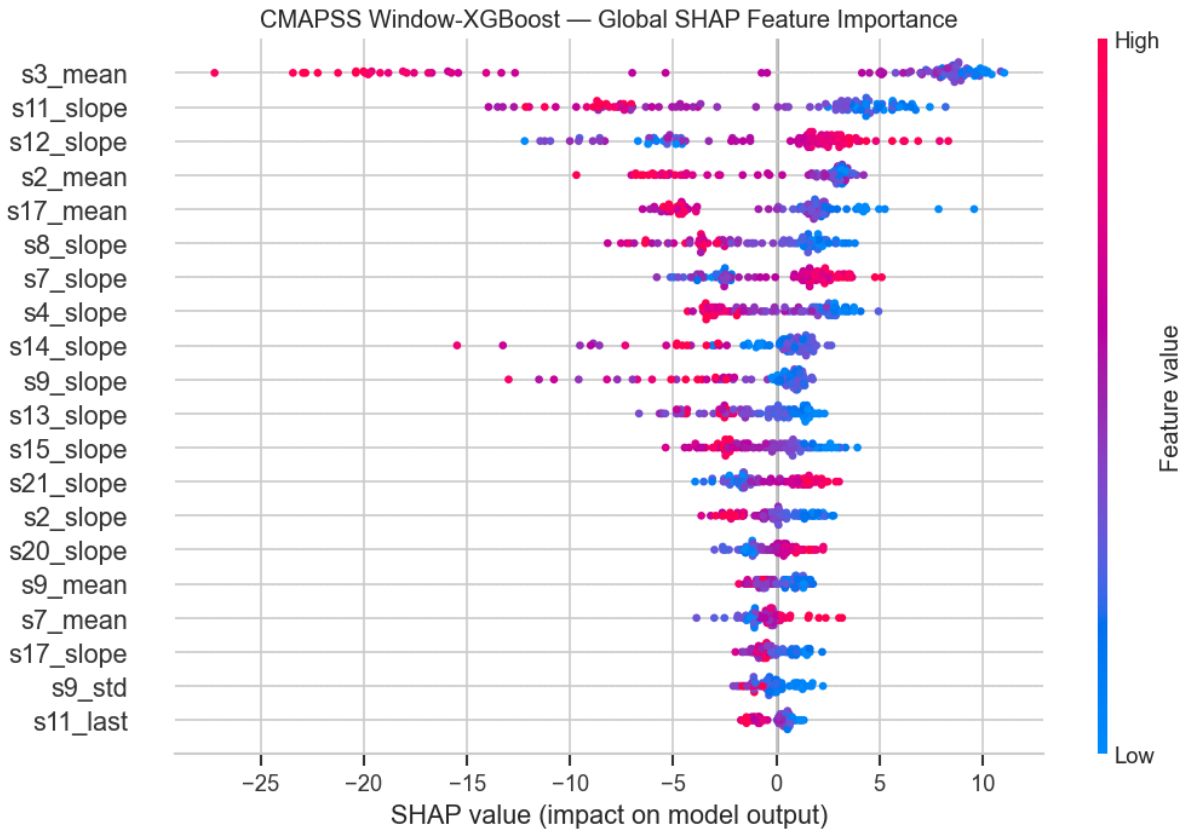
SHAP (SHapley Additive exPlanations) provides a game-theoretically grounded decomposition of each model's output into per-feature contributions. We apply SHAP TreeExplainer to the XGBoost models — the primary nonlinear tabular model for both domains — to answer two questions:

1. **Global importance**: Which sensors drive predictions most strongly across all examples?
2. **Local explanation**: For a single test example (one engine at critical RUL), how did each sensor contribute to the model's specific prediction?

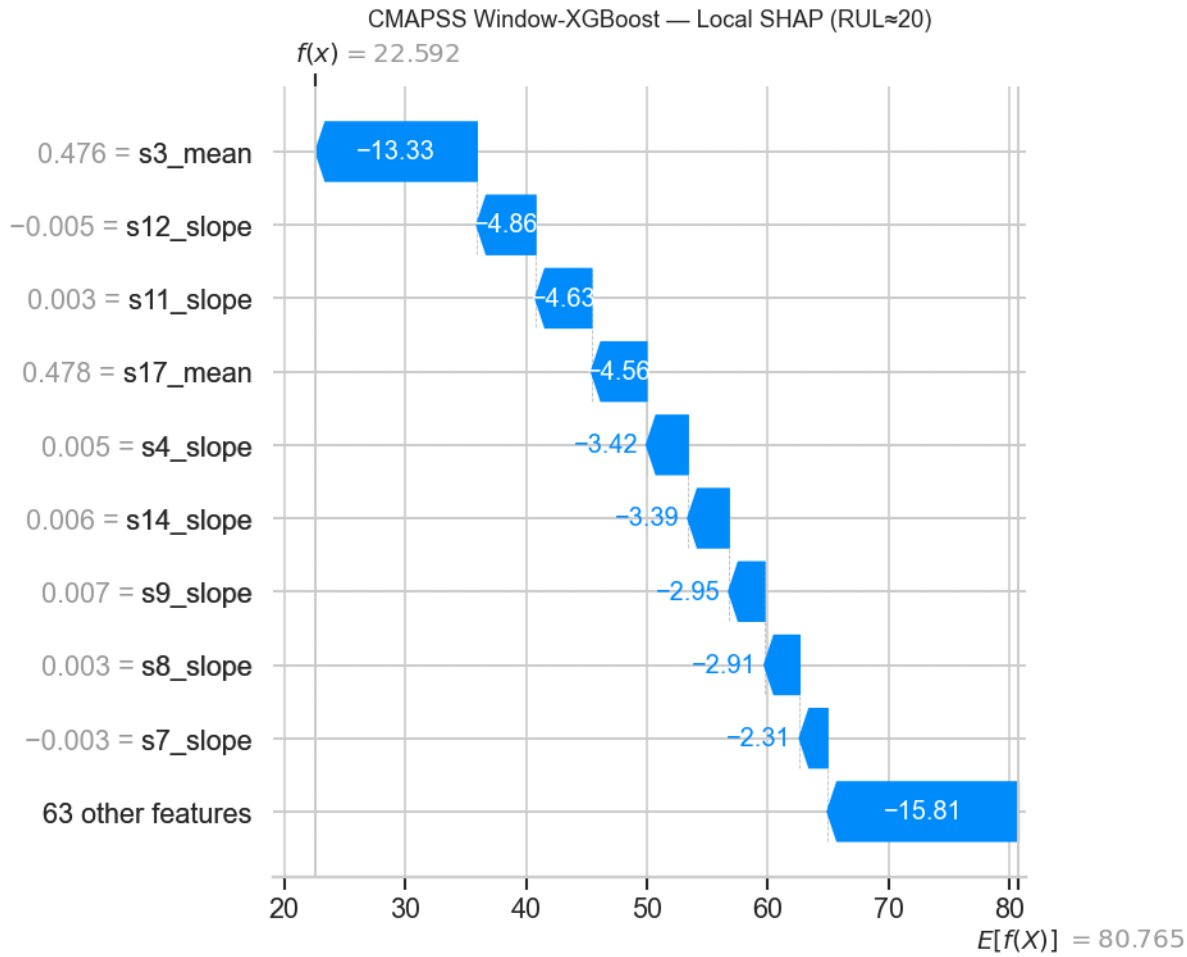
This analysis directly addresses deployment trust: maintenance engineers must understand *why* the model is raising an alert before they can act on it.

In [60]:
```
# ==================================================================
```

```
# SHAP — CMAPSS Window-XGBoost (global & local)
# ==================================================================

stat_names = ["last", "mean", "std", "slope"]
win_feat_names = [f"{f}_{s}" for f in FEAT_COLS for s in stat_names]

explainer_cmapss = shap.TreeExplainer(xgb_win)
shap_vals_cmapss = explainer_cmapss.shap_values(X_wfeat_test)

# Global beeswarm
fig, ax = plt.subplots(figsize=(9, 6))
shap.summary_plot(shap_vals_cmapss, X_wfeat_test,
                  feature_names=win_feat_names, show=False, plot_size=
plt.gca().set_title("CMAPSS Window-XGBoost — Global SHAP Feature Impor
plt.tight_layout()
plt.savefig("images/shap_cmapss_global.png", dpi=120, bbox_inches="tig
plt.show()

mean_shap = np.abs(shap_vals_cmapss).mean(axis=0)
shap_rank = (pd.DataFrame({"Feature": win_feat_names, "Mean |SHAP|": m
               .sort_values("Mean |SHAP|", ascending=False))
print("=== CMAPSS — Top-10 features by mean |SHAP| ===")
print(shap_rank.head(10).to_string(index=False))

# Local: engine nearest RUL=20
eol_idx = int(np.argmin(np.abs(y_wfeat_test - 20)))
print(f"\nLocal explanation — index {eol_idx}, true RUL={y_wfeat_test[
      f"predicted={pred_win_test[eol_idx]:.1f}")

shap.waterfall_plot(
    shap.Explanation(
        values=shap_vals_cmapss[eol_idx],
        base_values=explainer_cmapss.expected_value,
        data=X_wfeat_test[eol_idx],
        feature_names=win_feat_names,
    ),
    show=False,
)
plt.title(f"CMAPSS Window-XGBoost — Local SHAP (RUL≈{y_wfeat_test[eol_
plt.tight_layout()
plt.savefig("images/shap_cmapss_local.png", dpi=120, bbox_inches="tigh
plt.show()
```



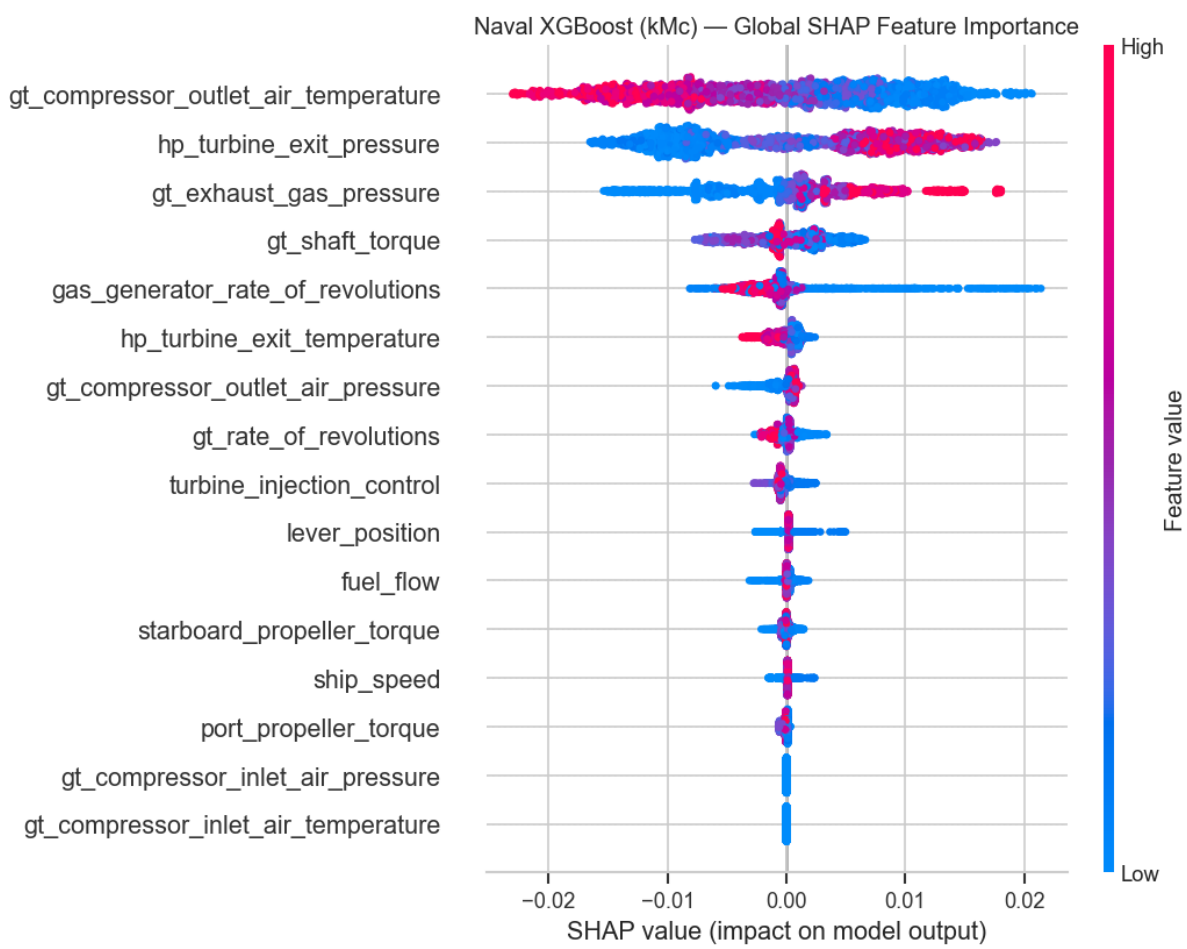

```
=== CMAPSS — Top-10 features by mean |SHAP| ===
  Feature  Mean |SHAP|
  s3_mean    10.892962
s11_slope     5.720564
s12_slope     4.118238
  s2_mean     3.473572
 s17_mean     3.286206
 s8_slope     2.697567
 s7_slope     2.540538
 s4_slope     2.335183
s14_slope     2.042875
 s9_slope     1.905003

Local explanation — index 23, true RUL=20, predicted=22.6
```



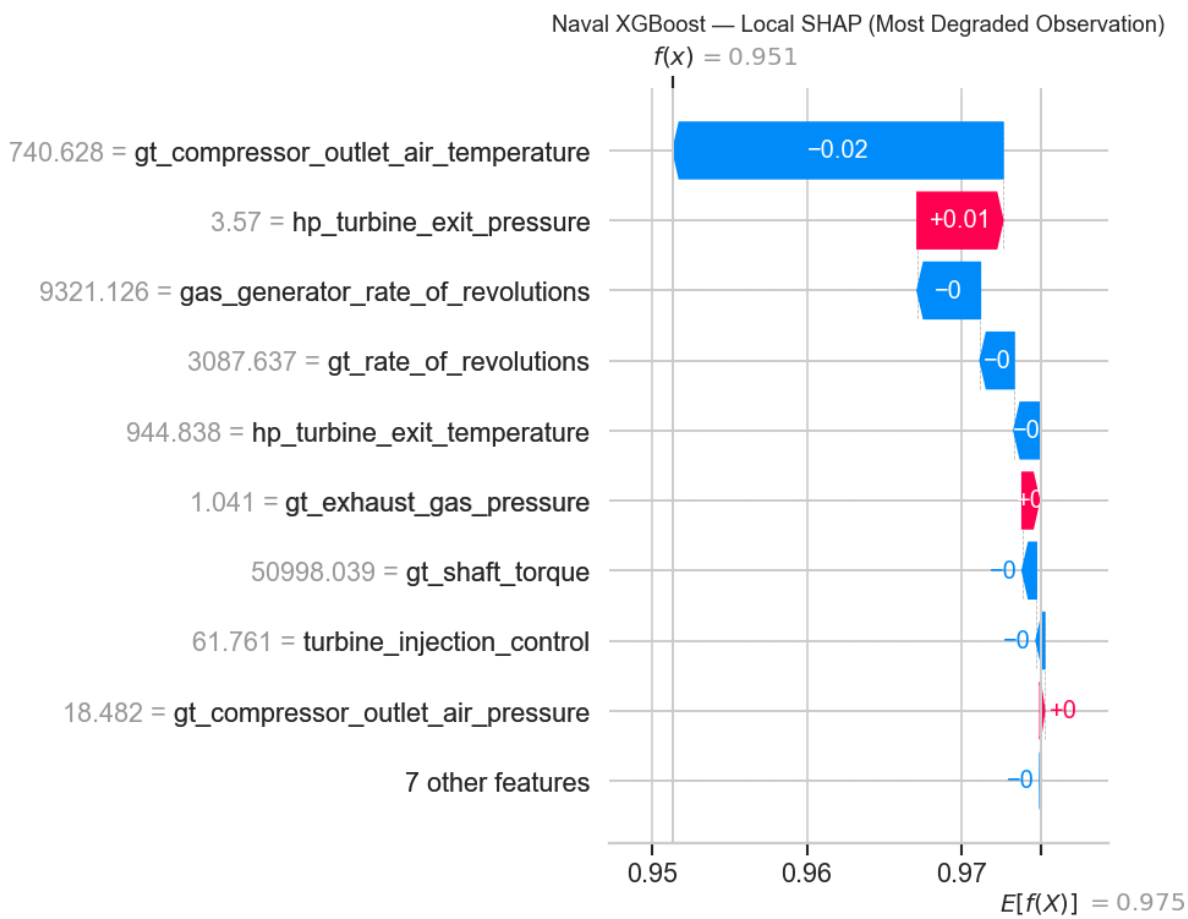
In [61]:

```
# ==================================================================
# SHAP — NAVAL XGBoost (kMc target)
# Purpose: Reveal which operational parameters drive compressor decay
#          predictions. Particularly important given the near-zero lin
#          correlations found in EDA — SHAP shows *how* nonlinear inte
#          are used to recover the degradation signal.
# ==================================================================

explainer_nav = shap.TreeExplainer(xgb_nav_kMc)
shap_vals_nav = explainer_nav.shap_values(X_nav_test)

# ── Global beeswarm ──────────────────────────────────────────────
fig, ax = plt.subplots(figsize=(9, 7))
shap.summary_plot(shap_vals_nav, X_nav_test,
                  feature_names=naval_x.columns.tolist(),
                  show=False, plot_size=None)
plt.gca().set_title("Naval XGBoost (kMc) — Global SHAP Feature Importa
plt.tight_layout()
plt.savefig("images/shap_naval_global.png", dpi=120, bbox_inches="tigh
plt.show()

# Mean |SHAP| ranking — Naval kMc
mean_shap_nav = np.abs(shap_vals_nav).mean(axis=0)
shap_rank_nav = pd.DataFrame({
    "Feature": naval_x.columns.tolist(),
```

```
    "Mean |SHAP|": mean_shap_nav,
}).sort_values("Mean |SHAP|", ascending=False)
print("=== Naval (kMc) — Feature importance by mean |SHAP| ===")
print(shap_rank_nav.to_string(index=False))

# —— Local waterfall — single observation at lowest kMc (most degraded
most_degraded_idx = int(np.argmin(y_nav_test[:, 0]))
print(f"\nLocal explanation — most degraded test obs (kMc={y_nav_test[
      f"predicted={pred_xgb_nav_test[most_degraded_idx, 0]:.4f})")

fig, ax = plt.subplots(figsize=(9, 5))
shap.waterfall_plot(
    shap.Explanation(
        values=shap_vals_nav[most_degraded_idx],
        base_values=explainer_nav.expected_value,
        data=X_nav_test[most_degraded_idx],
        feature_names=naval_x.columns.tolist(),
    ),
    show=False,
)
plt.title("Naval XGBoost — Local SHAP (Most Degraded Observation)", fo
plt.tight_layout()
plt.savefig("images/shap_naval_local.png", dpi=120, bbox_inches="tight
plt.show()
```



```
=== Naval (kMc) — Feature importance by mean |SHAP| ===
                              Feature  Mean |SHAP|
gt_compressor_outlet_air_temperature     0.008361
            hp_turbine_exit_pressure     0.008308
             gt_exhaust_gas_pressure     0.004804
                     gt_shaft_torque     0.002612
   gas_generator_rate_of_revolutions     0.002428
         hp_turbine_exit_temperature     0.000867
   gt_compressor_outlet_air_pressure     0.000691
             gt_rate_of_revolutions     0.000643
          turbine_injection_control     0.000494
                     lever_position     0.000336
                          fuel_flow     0.000294
         starboard_propeller_torque     0.000255
                        ship_speed     0.000176
               port_propeller_torque     0.000145
 gt_compressor_inlet_air_temperature     0.000000
    gt_compressor_inlet_air_pressure     0.000000

Local explanation — most degraded test obs (kMc=0.9500, predicted=0.951
4)
```



In [62]:
```
# ==================================================================
# GENERALISATION CHECK — All CMAPSS Subsets (FD001–FD004)
# Uses Window-XGBoost (no PyTorch). Trains one model per subset.
# ==================================================================
```

```
generalisation_results = []

for subset in ["FD001", "FD002", "FD003", "FD004"]:
    tr, te = load_cmapss(subset=subset, raw_dir=CMAPSS_DIR, rul_clip=R
    fc = [c for c in tr.columns if c not in ("unit_id", "cycle", "rul"

    df_tr, df_val = split_engines(tr, val_fraction=0.20, random_state=
    Xtr, ytr = make_window_features(df_tr,  fc, window_size=WINDOW, st
    Xva, yva = make_window_features(df_val, fc, window_size=WINDOW, st
    Xte, yte = get_test_window_features(te, fc, window_size=WINDOW)

    r_mean = evaluate_cmapss(yte, np.full_like(yte, ytr.mean()),
                             name="Mean", split=f"{subset} Test")

    model = xgb.XGBRegressor(
        n_estimators=300, max_depth=6, learning_rate=0.05,
        subsample=0.8, colsample_bytree=0.8,
        random_state=RANDOM_SEED, eval_metric="rmse",
        early_stopping_rounds=20, verbosity=0,
    )
    model.fit(Xtr, ytr, eval_set=[(Xva, yva)], verbose=False)
    preds = model.predict(Xte)
    r_win = evaluate_cmapss(yte, preds, name="Window-XGBoost", split=f

    print(f"{subset}  Mean RMSE={r_mean['RMSE']:.2f}  |  "
          f"Window-XGBoost RMSE={r_win['RMSE']:.2f}  R²={r_win['R²']:.
          f"NASA={r_win['NASA Score']:.1f}")
    generalisation_results.extend([r_mean, r_win])

print("\n=== Cross-Subset Summary ===")
gen_df = pd.DataFrame(generalisation_results)
gen_df.index = gen_df["Split"] + " — " + gen_df["Model"]
print(gen_df.drop(columns=["Split", "Model"]).to_string())
```

```
FD001  Mean RMSE=41.89  |  Window-XGBoost RMSE=13.35  R²=0.8969  NASA=2
48.9
FD002  Mean RMSE=53.78  |  Window-XGBoost RMSE=29.97  R²=0.6894  NASA=1
2831.1
FD003  Mean RMSE=43.70  |  Window-XGBoost RMSE=14.85  R²=0.8712  NASA=3
75.2
FD004  Mean RMSE=54.59  |  Window-XGBoost RMSE=31.48  R²=0.6667  NASA=1
0280.2

=== Cross-Subset Summary ===
                              RMSE     MAE      R²  NASA Score
FD001 Test — Mean           41.891  36.139 -0.0162     19095.5
FD001 Test — Window-XGBoost 13.346   9.998  0.8969       248.9
FD002 Test — Mean           53.780  45.292 -0.0000    125649.3
FD002 Test — Window-XGBoost 29.973  21.120  0.6894     12831.1
FD003 Test — Mean           43.701  36.985 -0.1145     39890.0
FD003 Test — Window-XGBoost 14.854  11.298  0.8712       375.2
FD004 Test — Mean           54.585  46.488 -0.0022    146267.4
FD004 Test — Window-XGBoost 31.478  23.279  0.6667     10280.2
```